当代人工智能实验课：实验二 - 搜索算法

助教: 余婧（51285903044@stu.ecnu.edu.cn）张珈鸣（51285903058@stu.ecnu.edu.cn)

日期: 2026年10月15日

提交截止日期：2026年11月12日23:59:59

提交方式后续在群里公布

实验目标

大家好！欢迎来到我们的第二个实验：搜索算法。本次实验的目标是：掌握课上所讲述的搜索算法，并且通过代码实现。

本次实验主要需要同学们实现A* 算法以及 alpha-beta 剪枝算法。所有题目的测试用例已给出，请按照测试用例的格式设计代码的输入输出。

算法题目

题目一 迷宫寻路

在一个给定的二维网格中，请你通过A*算法找到起点S到终点E的最短路径。网格中包含可通过的路径.以及不可通过的障碍物X。

你的程序需要处理以下情况：

1. 找到并返回最短路径的坐标序列。

2. 如果起点和终点之间不存在任何路径，则返回空列表。

规则：

- 移动：只能上下左右四个方向移动。

- 代价：每移动一格的代价（g-cost）为 1。

提示：

- 启发函数 (Heuristic): 可以考虑使用曼哈顿距离作为启发函数 h(n)。

输入格式：

- 一个Tuple[int, int]表示起点坐标(row, col)

算法题目（续）

- 一个Tuple[int, int]表示终点坐标(row, col)

- 一个List[List[str]]或者等效的二维数组表示一个地图

输出格式：

- 一个List[Tuple[int, int]]，按顺序包含从起点到终点的最短路径上所有坐标，如果无解，返回一个空列表[]

测试用例：

输入：

```
(0,0)
(3,4)
[
  ['S', '.', '.', 'X', '.'],
  ['.', 'X', '.', '.', '.'],
  ['.', 'X', '.', 'X', '.'],
  ['.', '.', '.', 'X', 'E'],
  ['X', 'X', '.', '.', '.']
]
```

算法题目（续）

预期输出：

```
[(0,0), (0,1), (0,2), (1,2), (1,3), (1,4), (2,4), (3,4)]
```

(注意: 可能存在多条相同长度的最短路径，输出任意一条即可)

题目二： 加权地形

现在地图不再是简单的可通过/不可通过，而是包含了不同类型的地形，移动到不同地形上的代价也不同。请你实现一个 A* 算法，计算从起点 S 到终点 E 的最小移动代价。

规则:

- 地图由代表移动代价的数字组成。例如，1 代表平原（代价为1），3 代表沼泽（代价为3）。
- 移动：同样只能上下左右四个方向移动。
- 代价：从节点 A 到相邻节点 B 的代价，等于进入节点 B 的地形权值。

算法题目（续）

提示：

- 这时候使用曼哈顿距离仍然是一个可采纳的启发函数吗？

输入格式：

- 一个Tuple[int, int]表示起点坐标(row, col)
- 一个Tuple[int, int]表示终点坐标(row, col)
- 一个List[List[str]]或者等效的二维数组表示一个地图

输出格式：

- 一个int，代表从起点到终点的最小总代价。如果无解，则返回 -1。

测试用例：

算法题目（续）

输入：

```
(0, 0)
(4, 4)
[
  [S, 1, 1, 8, 1],
  [1, 8, 3, 1, 1],
  [1, 8, 1, 8, 1],
  [1, 1, 1, 8, 1],
  [8, 8, 1, 1, E]
]
```

预期输出：7

最短代价路径为： (0,0)->(1,0)->(2,0)->(3,0)->(3,1)->(3,2)->(4,2)->(4,3)->(4,4)，总代价为7（最后一步没有代价，即可以将E视为0）

算法题目（续）

题目三： 极大极小博弈树的最佳得分

请你通过Alpha-beta剪枝求解一个极大极小博弈树的最佳的分。你需要编写一个函数，该函数接收一个树的根节点，返回对于MAX玩家而言最佳的结果。

规则：

- 树的结构已经为你定义好。非叶子节点有 children 属性，叶子节点有 value 属性。
- 你将从根节点开始，根节点是 MAX 玩家。
- MAX 玩家的目标是最大化得分，MIN 玩家的目标是最小化得分。
- 评估子节点的顺序应为从左到右。

数据结构定义 (Python 示例):

```
class TreeNode:
    def __init__(self, value=None, children=None):
        self.value = value  # 叶子节点的得分
        self.children = children if children is not None else [] # 子节点列表
```

算法题目（续）

测试用例

如下结构的树

```
      (MAX)
      /   \
    (MIN) (MIN)
    / \   / \
(MAX) (MAX) (MAX)
 / \   / \   / \
3   5 6   9 1   2
```

算法题目（续）

输入样例：

```
{"tree_structure": {
        "is_maximizing_player": true,
        "children": [
          {
            "is_maximizing_player": false,
            "children": [
              {
                "is_maximizing_player": true,
                "children": [
                  {"value": 3},
                  {"value": 5}
                ]
              },
              {
                "is_maximizing_player": true,
                "children": [
                  {"value": 6},
                  {"value": 9}
                ]
              }
            ]
```

算法题目（续）

代码续

```
          },
          {
            "is_maximizing_player": false,
            "children": [
              {
                "is_maximizing_player": true,
                "children": [
                  {"value": 6},
                  {"value": 9}
                ]
              },
              {
                "is_maximizing_player": true,
                "children": [
                  {"value": 1},
                  {"value": 2}
                ]
              }
            ]
          }
        ]
      }
}
```

算法题目（续）

构建代码示例：

```
# 叶子节点
leaf1 = TreeNode(value=3)
leaf2 = TreeNode(value=5)
leaf3 = TreeNode(value=6)
leaf4 = TreeNode(value=9)
leaf5 = TreeNode(value=1)
leaf6 = TreeNode(value=2)

# 中间层 (MAX)
max1 = TreeNode(children=[leaf1, leaf2])
max2 = TreeNode(children=[leaf3, leaf4])
max3 = TreeNode(children=[leaf5, leaf6])

# 中间层 (MIN)
min1 = TreeNode(children=[max1, max2])
min2 = TreeNode(children=[max2, max3])

# 根节点 (MAX)
root = TreeNode(children=[min1, min2])
```

算法题目（续）

预期输出：5

从左下开始进行选取，向上一层为MAX玩家，因此选取其子节点的最大值，分别为：5，9，2

向上一层为MIN玩家，因此选取其子节点最小值，分别为：5，2

根节点为MAX玩家，因此选取其子节点最大值，5

最终预期输出为5

算法题目（续）

题目四：竞争环境下的宝藏分配（综合题）

两名玩家 A（MAX）与 B（MIN）轮流争夺四个宝藏。每次选取一个尚未获得的宝藏，玩家会沿最小代价路径移动到该位置。请结合 A* 与 Alpha-beta 剪枝，求 A 的最优首步。

地图与宝藏价值：

```
     0   1   2   3   4   5   6
0    A   1   1   X   2   1   T1
1    1   X   6   X   1   X   1
2    1   1   T2  1   1   1   1
3    X   1   X   8   X   1   X
4    1   1   1   1   T3  1   1
5    1   X   9   X   1   X   1
6    T4  1   1   X   1   1   B
```

宝藏	位置(row,col)	价值
T1	(0, 6)	**9**
T2	(2, 2)	**6**
T3	(4, 4)	**7**
T4	(6, 0)	**7**

规则：

- X 为障碍物，数字表示进入该格的代价；A、B、T 所在格代价为 1，只能上下左右移动。
- 行动顺序为 A → B → A → B，共四回合；每回合只选一个宝藏，获取后更新玩家位置。
- 经过其他宝藏不自动收集；已获取宝藏的位置仍可通行；另一玩家不阻挡寻路。

算法题目（续）

收益计算：

设宝藏 Ti 的价值为 Vi，d(p,Ti) 为从当前位置 p 到 Ti 的最小移动代价。

$$Gain(p, Ti) = Vi - d(p, Ti) \qquad U = Score_A - Score_B$$

A 选择使 U 最大的策略，B 选择使 U 最小的策略；到达搜索深度或宝藏取完时，以当前 U 作为评价值。

任务要求：

- 1. 使用 A* 计算 A、B、T1、T2、T3、T4 两两之间的最小移动代价，输出 6×6 距离矩阵。
- 2. 自行设计博弈状态，至少记录双方位置、剩余宝藏、累计收益和当前玩家。
- 3. 分别使用 Minimax 和 Alpha-beta，在深度 H=2、H=4 时计算 A 的最优首步和最优收益差。

测试用例与预期输出：

输入：本题给定地图及宝藏价值

```
H = 2   =>  {"best_target": "T2", "best_value": -1}

H = 4   =>  {"best_target": "T3", "best_value":  3}
```

实验报告要求

由于题目已给出测试用例与预期结果，本次实验重点考察算法设计、对照实验及结果分析。仅展示运行截图或复述算法定义不足以取得高分。

以下为实验报告的建议内容；具体分值请以第17页评分细则为准。

1. 算法设计与实验设置

说明 A* 的状态、g/h/f 与优先队列，以及 Minimax、Alpha-Beta 的递归和剪枝逻辑。

2. 题目一：迷宫寻路

解释最短路径的形成过程；比较 h=0 与曼哈顿距离的搜索节点数并分析差异。

3. 题目二：加权地形

展示路径代价的累计过程；结合终点 E 的进入代价规定，讨论启发函数可采纳性。

4. 题目三：Alpha-Beta 剪枝

说明 Minimax 数值回传与剪枝依据；比较不同子节点访问顺序的搜索开销。

5. 题目四：综合题

给出距离矩阵与状态表示；分析 H=2、H=4 的决策差异及不同访问顺序的剪枝效果。

6. 自设对照实验与拓展

设计额外输入、参数或搜索策略，记录实验现象并解释原因；创新优化须有证据。

7. 图表展示与独立思考

用表格、示意图和具体数据支撑结论，重点说明“为什么”，避免只罗列输出。

评分细则

总分 100 分：代码实现 30 分 | 算法性能 10 分 | 实验报告 50 分 | 创新与拓展 10 分

项目	权重	评分细则
1. 代码实现（30分）		
- 算法实现完整	**10**	实现 A*、Minimax 与 Alpha-Beta，满足功能和输入输出要求
- 搜索过程与状态管理	**10**	正确处理启发函数、路径代价、状态更新与剪枝条件
- 代码规范与可复现性	**10**	结构清晰，注释适当；提供 README，结果可复现
2. 算法性能（10分）		
- 搜索效率评估	**5**	用节点数、时间等指标分析启发函数与搜索策略的影响
- 剪枝效率评估	**5**	比较 Minimax、Alpha-Beta 开销及访问顺序与剪枝效果
3. 实验报告（50分）		
- 报告结构与条理	**10**	包含目标、设计、设置、结果、分析和总结；逻辑清晰
- 算法原理与实现思路	**10**	说明启发函数、路径代价、博弈搜索及综合题状态建模
- 结果展示与对照分析	**15**	用图表和对照实验分析启发函数、深度与访问顺序
- 深入分析与独立思考	**15**	结合数据和搜索过程解释算法行为与性能差异
4. 创新与拓展（10分）		
- 算法优化与改进	**5**	尝试改进启发函数、节点排序、缓存或其他合理策略
- 拓展实验与探索分析	**5**	自设场景，分析适用条件、效率瓶颈或特殊情况

说明：给定测试样例的正确性是基础要求；算法性能与创新拓展须在报告中提供实验数据及分析依据。
性能不按速度或节点数量简单排名；创新不要求全新算法，由助教依据完成质量酌情评分。